



UNIVERSIDAD DE MÁLAGA



PROYECTO: ROBOT MÓVIL DIFERENCIAL

Laboratorio de robótica
4º GIERM

Nombre: María José López Pérez
DNI: 26927018K

Índice

1. Introducción	3
2. Hardware	4
2.1. Componentes del robot	4
2.2. Esquema de conexionado	6
3. Software	8
3.1. Planteamiento general	8
3.1.1. Topics	8
3.1.2. Fórmulas	9
3.2. Programas desarrollados	10
3.2.1. velocidad	13
3.2.2. posicion	13
3.2.3. posicion_control	13
3.2.4. circulo	13
3.2.5. follow	13
3.2.6. sensor_simple	13
3.2.7. sensor	14
3.2.8. mando	14
3.3. ¿Cómo arrancar el robot?	14
4. Resultados	16

Índice de figuras

1.	Robot móvil diferencial	3
2.	Materiales necesarios	4
3.	Componentes adicionales	5
4.	Conexionado	6
5.	Movimiento robot móvil	9
6.	Archivos .hpp y .cpp	11
7.	Estructura .cpp	12
8.	Mando Nintendo Switch	14
9.	Código QR Repositorio GitHub	16

1. Introducción

Con este proyecto se pretende construir desde cero un robot móvil diferencial. Para ello, será necesario diseñar la carcasa, elegir los componentes electrónicos, realizar la identificación de los motores y desarrollar distintos programas que permitan verificar su correcto funcionamiento.



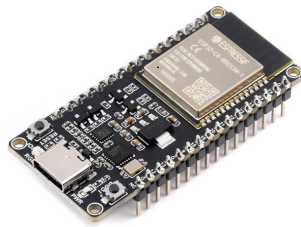
Figura 1: Robot móvil diferencial

2. Hardware

2.1. Componentes del robot

Los principales materiales que se van a utilizar son:

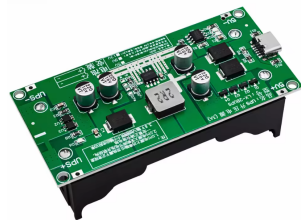
- Microcontrolador ESP32.
- Driver de potencia L298N.
- Dos motores de corriente continua JGA25-370
- Tres sensores de ultrasonidos HC-SR04P.



(a) Microcontrolador ESP32



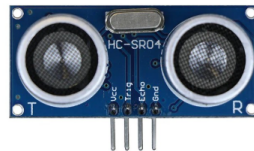
(b) Driver de potencia



(c) Portapilas



(d) Motor JGA25-370



(e) Sensor HC-SR04P

Figura 2: Materiales necesarios

También se va a incorporar un interruptor que permitirá cortar la corriente que le llega al driver desde el portapilas, y por tanto a los motores, y un mini voltímetro para medir el voltaje que le llega a los motores.



(a) Interruptor



(b) Mini voltímetro



(c) Componentes integrados

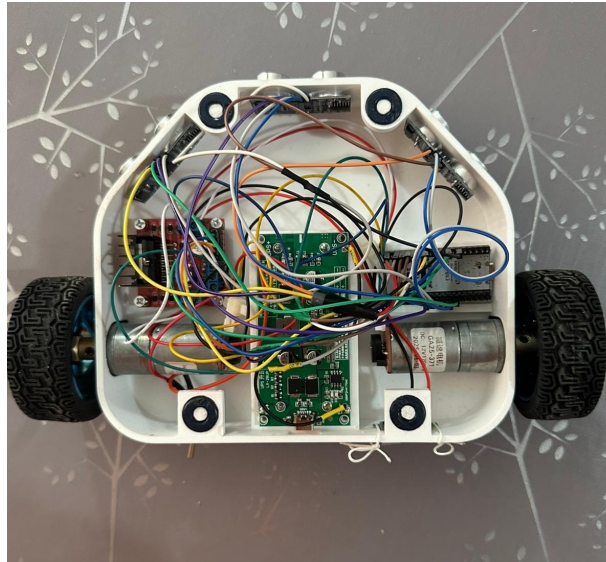
Figura 3: Componentes adicionales

Además, en la parte delantera lleva una "rueda loca" para que mantenga el equilibrio y pueda andar sin problemas.

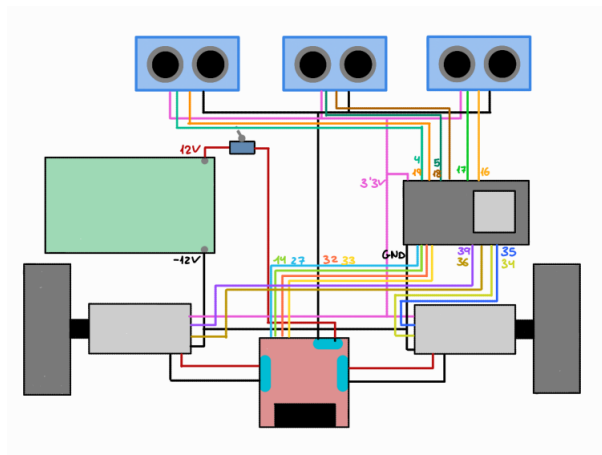
El diseño de la carcasa se ha realizado con el software "Onshape" y se ha impreso con una impresora 3D.

2.2. Esquema de conexionado

A continuación, se puede observar todo el cableado que hay en el interior del robot. Para poder visualizarlo mejor se ha realizado un esquema.



(a) Cableado robot



(b) Esquema

Figura 4: Conexionado

Donde:

- Todos los elementos comparten la misma referencia de tierra.
- El driver de potencia se alimenta con 12 V .
- La alimentación del motor se conecta al driver y en medio se encuentra el interruptor.
- La señal PWM le llega a un motor desde los pines 32 y 33 y al otro motor desde los pines 14 y 27 del microcontrolador, pasando por el driver de potencia.

- Los encoders necesitan 3.3V para funcionar.
- Los encoders A y B se leen a través de los pines 39, 36, 35 y 34.
- Cada sensor de ultrasonidos requiere dos pines de conexión: uno destinado al envío de la señal de disparo (trigger) y otro para la recepción del eco reflejado (echo), a partir del cual se calcula la distancia. Como trigger se usan los pines 4, 5 y 17 y como echo, el 19, 18 y 16.

3. Software

3.1. Planteamiento general

El microcontrolador ejecuta un programa desarrollado en Arduino en el que se ha implementado un controlador PID para cada motor, permitiendo regular su velocidad de forma precisa. Los parámetros de cada uno se han obtenido siguiendo la práctica 3. Además, se encarga de la lectura de los encoders y de los sensores de ultrasonidos.

En el ordenador se ejecutarán nodos de ROS 2 y para permitir la comunicación es necesario instalar un agente de micro-ROS en dicho equipo e integrar micro-ROS en la ESP32. Esta comunicación se establece a través de la red WiFi, por lo que la ESP32 debe estar conectada a la misma red que el ordenador que ejecuta ROS 2.

3.1.1. Topics

- `/cmd_vel`: El nodo de ROS 2 publica la velocidad lineal deseada en un mensaje tipo "Twist" y la ESP32, suscrita al topic, la recibe para pasarla velocidad angular y poder actuar sobre los motores con una señal PWM.
- `/vel_pub_left`: La ESP32 publica la velocidad angular real del motor izquierdo, esta se calcula cada 10 ms con una interrupción del timer.
- `/vel_pub_right`: La ESP32 publica la velocidad angular real del motor derecho, esta se calcula cada 10 ms con una interrupción del timer.
- `/pose_pub`: En el "loop" tenemos una función que se encarga de calcular la pose por odometría, es decir, por los pulsos del encoder, de esta manera podemos saber las coordenadas "x" e "y" y la orientación del robot. La ESP32 publica un mensaje tipo "Pose" donde rellena los campos "position.x", "position.y" y "orientation.z".
- `/sensors_pub`: La ESP32 publica un vector con los valores de los sensores, el primer valor corresponde al sensor central, el segundo al derecho y el tercero al izquierdo.

3.1.2. Fórmulas

Para que el robot gire correctamente, es necesario calcular su modelo cinemático inverso, que permite determinar las velocidades individuales de cada rueda en función del movimiento deseado del robot (traslación y rotación)

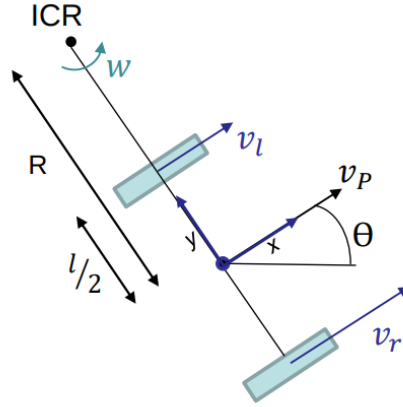


Figura 5: Movimiento robot móvil

Donde:

- **ICR:** Centro Instantáneo de Rotación.
- v_p : Velocidad lineal del robot.
- w : Velocidad angular del robot.
- R : Radio de giro.
- $\frac{l}{2}$: Medida del robot, en mi caso 0.115 cm.
- v_l : Velocidad rueda izquierda.
- v_r : Velocidad rueda derecha.

La velocidad que debe llevar cada rueda se calcula mediante:

$$v_r = v_P + w \cdot \frac{l}{2}$$
$$v_l = v_P - w \cdot \frac{l}{2}$$

Estas ecuaciones están integradas en el código de arduino de manera que, a partir de la velocidad que llega a través del topic `/cmd_vel` desde el nodo de ROS2, se calcula la velocidad de cada rueda para después calcular la señal PWM.

Por otra parte, para el cálculo de la odometría se necesitan las siguientes fórmulas:

$$s_r = 2 \cdot \pi \cdot r \cdot \frac{n_r}{N_{rev}}$$
$$s_l = 2 \cdot \pi \cdot r \cdot \frac{n_l}{N_{rev}}$$
$$x = \frac{s_r + s_l}{s_r - s_l} \cdot \frac{l}{2} \cdot \sin \frac{s_r - s_l}{l}$$
$$y = \frac{s_r + s_l}{s_r - s_l} \cdot \frac{l}{2} \cdot [1 - \cos \frac{s_r - s_l}{l}]$$
$$\theta = \frac{s_r - s_l}{l}$$

Donde:

- s_r y s_l : Arco recorrido por cada rueda
- n_r y n_l : Número de pulsos de encoder
- N_{rev} : Número de pulsos por vuelta.
- r : Radio de las ruedas
- x : Posición en x.
- y : Posición en y.
- θ : Orientación.

Estas ecuaciones están integradas en el programa Arduino, donde se calcula la posición del robot y se envía al nodo a través del topic /pose_pub

3.2. Programas desarrollados

Se van a explicar los nodos que se han desarrollado, pues el código de Arduino ya se ha explicado en 3.1. Planteamiento general. Se ha creado un paquete llamado "tarea1" en el que se alojan todos los nodos, cada uno corresponde a una funcionalidad.

Cuando entramos dentro del workspace nos encontramos con:

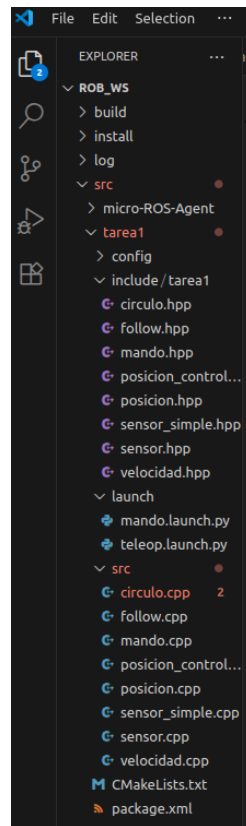
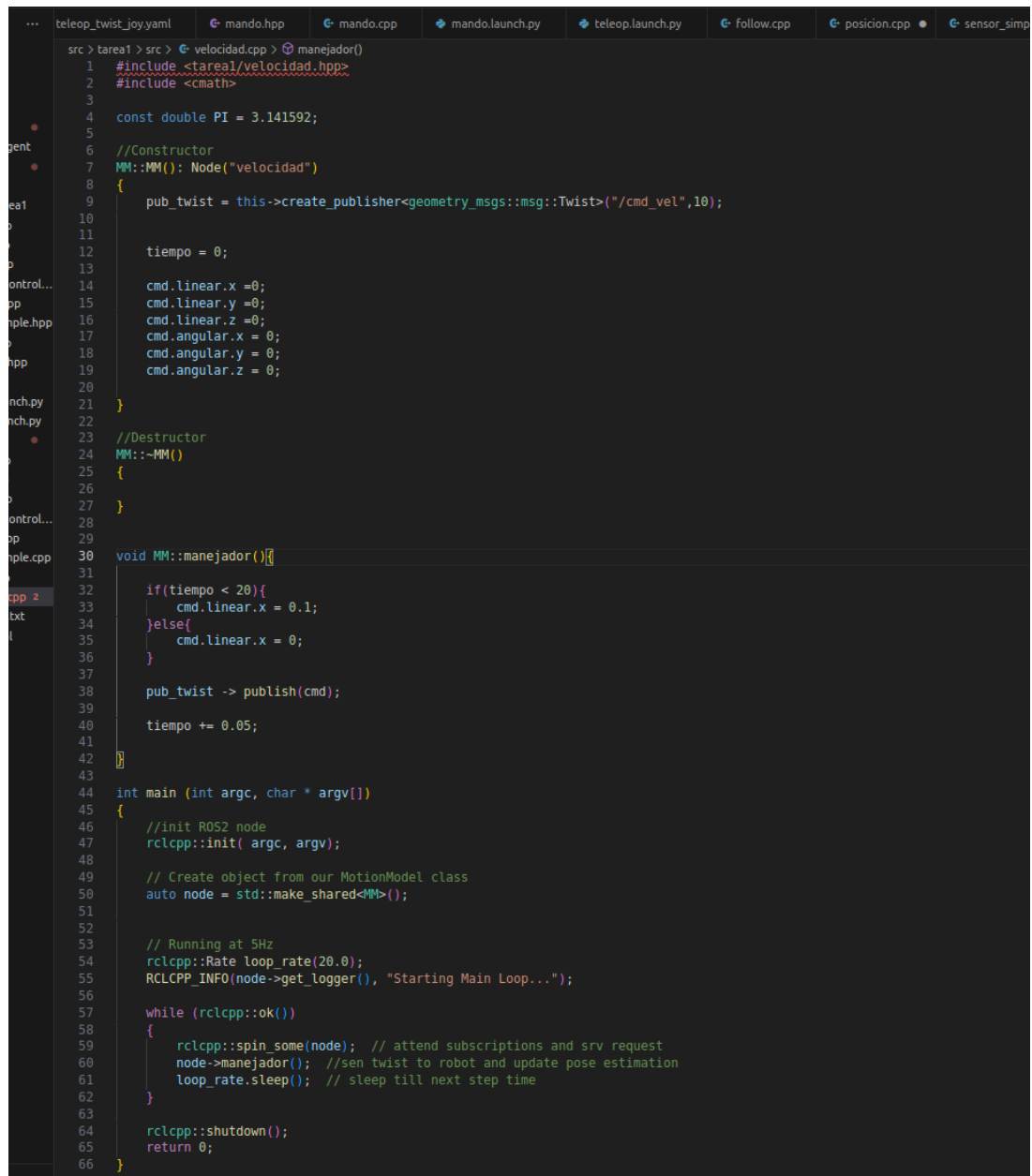


Figura 6: Archivos .hpp y .cpp

Cada archivo .cpp tiene la misma estructura:



```
src > tarea1 > src > velocidad.cpp > manejador()
1  #include <stareal/velocidad.hpp>
2  #include <cmath>
3
4  const double PI = 3.141592;
5
6  //Constructor
7  MM::MM(): Node("velocidad")
8  {
9      pub_twist = this->create_publisher<geometry_msgs::msg::Twist>("/cmd_vel",10);
10
11
12      tiempo = 0;
13
14      cmd.linear.x = 0;
15      cmd.linear.y = 0;
16      cmd.linear.z = 0;
17      cmd.angular.x = 0;
18      cmd.angular.y = 0;
19      cmd.angular.z = 0;
20  }
21
22  //Destructor
23  MM::~MM()
24  {
25  }
26
27
28
29
30 void MM::manejador(){}
31
32 if(tiempo < 20){
33     cmd.linear.x = 0.1;
34 }else{
35     cmd.linear.x = 0;
36 }
37
38 pub_twist -> publish(cmd);
39
40 tiempo += 0.05;
41
42 }
43
44 int main (int argc, char * argv[])
45 {
46     //init ROS2 node
47     rclcpp::init( argc, argv);
48
49     // Create object from our MotionModel class
50     auto node = std::make_shared<MM>();
51
52
53     // Running at 5Hz
54     rclcpp::Rate loop_rate(20.0);
55     RCLCPP_INFO(node->get_logger(), "Starting Main Loop...");
56
57     while (rclcpp::ok())
58     {
59         rclcpp::spin_some(node); // attend subscriptions and srv request
60         node->manejador(); //sen twist to robot and update pose estimation
61         loop_rate.sleep(); // sleep till next step time
62     }
63
64     rclcpp::shutdown();
65     return 0;
66 }
```

Figura 7: Estructura .cpp

- **Incluir archivo .hpp:** Archivo en el que se declara el nodo, las variables, publishers, suscribers y servicios que tendrá.
- **main:** Mantiene activo el nodo y ajusta los Herzios
- **Constructor y destructor del nodo:** Se inicializan las variables, se crean los publishers y suscribers y se configuran los servicios.
- **Manejador:** Es un método que se ha creado para enviar la velocidad lineal al microcontrolador.

- **callbacks:** Necesarios cuando tenemos un suscriber para gestionar los datos que llegan (En el ejemplo de la foto no hay)

3.2.1. velocidad

En este programa se envía la velocidad lineal de 0.1 m/s durante 20 segundos. Como se sabe la frecuencia con la que se ejecuta el código (loop_rate) se ha podido crear una variable de tiempo que vaya aumentando, y una vez que se llegue al tiempo requerido la velocidad que se envía es 0. (Vídeo: velocidad)

3.2.2. posicion

Se suscribe al topic /pose_pub y mientras que no se haya alcanzado $x == 2$ (2 metros) se envía una velocidad lineal de 0.25. Una vez alcanzada se envía 0. (Vídeo: posicion)

3.2.3. posicion_control

Al igual que antes, hasta que no se alcanza $x == 2$ se envía velocidad, la única diferencia es que en este caso se toma la orientación para poder realizar un control del mismo para que el robot vaya totalmente recto y se envía multiplicada por un parámetro como velocidad angular. (Vídeo: posicion_control)

3.2.4. circulo

Similar al programa "velocidad", se envía la velocidad angular de $\frac{2 \cdot \pi}{10}$ y se calcula la lineal en función del radio (0.4 metros en este caso) $w \cdot r$ del círculo. Durante el tiempo que se ha calculado que debe tardar en dar una vuelta se envían las velocidades, después la velocidad será nula. (Vídeo: circulo)

3.2.5. follow

En este programa se ha declarado un array con los puntos (1,1), (1.5, 1) y (1.5,2). Como esta suscrito al topic /pose_pub, sabe en todo momento donde está situado el robot, permitiendo calcular la distancia que falta para llegar al robot y la orientación, que dirá si es necesario girar o no. Se va recorriendo el array punto a punto, cuando está cercano al primer punto pasa al siguiente y así sucesivamente. Se envían las velocidades de cada rueda al microcontrolador, listas para ser traducidas a señal PWM.

En el vídeo parece que no sigue los puntos, pero el robot piensa que está en los puntos correctos, pues muestro por la terminal el mensaje del topic. (Vídeo: follow)

3.2.6. sensor_simple

Se suscribe al topic /sensors_pub y se evalúa cada uno de ellos. Si la distancia es menor de 10 centímetro en algunos de ellos se envía velocidad nula, de no serlo, se envía una velocidad lineal de 0.1 m/s. (Vídeo: sensor_simple)

3.2.7. sensor

Similar a "sensor_simple", se evalúa el dato de cada sensor, y si no hay obstáculos de frente, va recto, si se detecta alguno a lo lejos, se evalúan los sensores de los lados y se gira hacia el lado que hay más espacio. Si se detecta un obstáculo muy cerca, el robot se detiene indefinidamente, pues por sus medidas, cuando gire se chocará. (Vídeo: sensor)

3.2.8. mando

Permite manejar el robot con un mando tipo Xbox o Nintendo Switch.

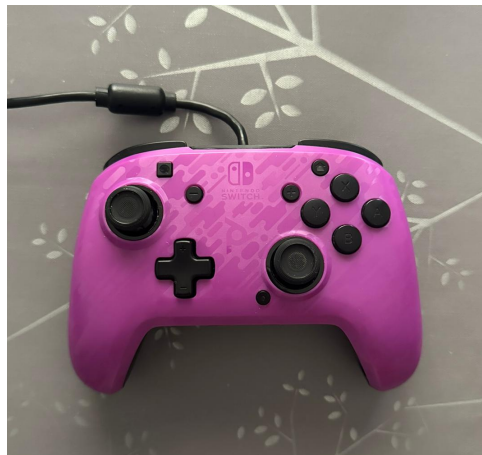


Figura 8: Mando Nintendo Switch

En este caso se lanza un archivo launch, que permite lanzar dos nodos al mismo tiempo. Se lanza el nodo `joy_node` (instalado en el ordenador) y `mando`.

- **joy_node:** Se encarga de transformar los controles del mando en un mensaje tipo joy y de publicarlo en un topic llamado `/joy`. Los botones pasan de 0 a 1 cuando se pulsan y los joysticks pueden tomar cualquier valor entre 0 y 1.
- **mando:** Se suscribe al topic `/joy` y escala los mensajes para enviar las velocidades lineales. Con el joystick izquierdo se maneja libremente el robot, tomando el eje y en velocidad linear y el eje x en velocidad angular. Los máximos que se pueden alcanzar son 0.15 m/s y 1 rad/s. Con los botones ZL Y ZR ("palancas" o "botones de disparo") el robot gira sobre sí mismo hacia la izquierda o hacia la derecha, respectivamente.

(Vídeo: mando)

3.3. ¿Cómo arrancar el robot?

Antes de seguir estos pasos es necesario tener la ESP32 programada con el código Arduino, tener instalado ROS2, el paquete de micro-ROS y tener el workspace en el ordenador.

Para poder usar el entorno de ROS2 es necesario cargar el mismo:

```
source /opt/ros/humble/setup.bash
```

También es necesario cargar el workspace:

```
source /rob_ws/install/setup.bash
```

En lugar de ejecutar estos dos comandos cada vez que se quiera usar programar, existe la opción de añadirlos al archivo `~/.bashrc` de manera que siempre que cada vez que se abre una terminal de Linux se ejecutará y no será necesario meterlos cada vez que se abra una nueva terminal.

El ordenador en el que se va a lanzar el agente de micro-ROS debe estar conectado a la misma red WiFi que la ESP32. Desde la terminal de Linux se ejecuta el comando:

```
ros2 run micro_ros_agent micro_ros_agent udp4 -port 8888
```

Una vez que la ESP32 se haya conectado al WiFi debe aparecer en pantalla que el cliente se ha conectado. Si no sale, pulsa RESET en el microcontrolador.

Ahora solo hay que lanzar el nodo para que se ejecute el programa deseado:

```
ros2 run tarea1 velocidad
```

```
ros2 run tarea1 posicion
```

```
ros2 run tarea1 posicion_control
```

```
ros2 run tarea1 circulo
```

```
ros2 run tarea1 follow
```

```
ros2 run tarea1 sensor_simple
```

```
ros2 run tarea1 sensor
```

```
ros2 launch tarea1 mando.launch.py
```


4. Resultados

Para facilitar la entrega de los vídeos demostrativos se ha creado un repositorio en GitHub llamado Robot Diferencial, en el que se han subido todos los código, las imágenes y los vídeos del robot.



Figura 9: Código QR Repositorio GitHub

Enlace al repositorio: <https://github.com/mariiaajoosee/Robot-Diferencial>

En cuanto a la nota que le asignaría al trabajo sería un 8.5, porque el control se puede plantear de manera distinta para que reaccione antes el robot, pues tanto en el programa del círculo como en el de la posición sobrepasa el objetivo unos centímetros. Sin embargo, se ha conseguido que micro-ROS tenga un gran papel dentro del proyecto, pues era la finalidad de esta asignatura, y además, se han implementado dos tareas opcionales y se ha añadido al robot el mini voltímetro y el interruptor. También se tiene que tener en cuenta el trabajo que supone hacer la identificación de los motores y el código para traducir la velocidad en pulso PWM, que, aunque ya estaba hecho de prácticas anteriores, era necesario adaptarlo y ha supuesto un gran trabajo, pues había que "dar con la tecla" para que el programa no se interrumpiera debido a algún fallo interno de algún tipo de variable usada. Por último, puede contar positivamente el diseño de la carcasa del robot, que aunque ha sido necesaria el uso de una batería externa para alimentar la ESP32 y no se ha podido mostrar bien en los vídeos y delante del profesor, la idea era simular un robot de juguete el cual incluye tapa, un agujero para cargarlo, y un hueco en la parte inferior para poner las pilas.